Architecture Explained — AnythingLLM Administration Console

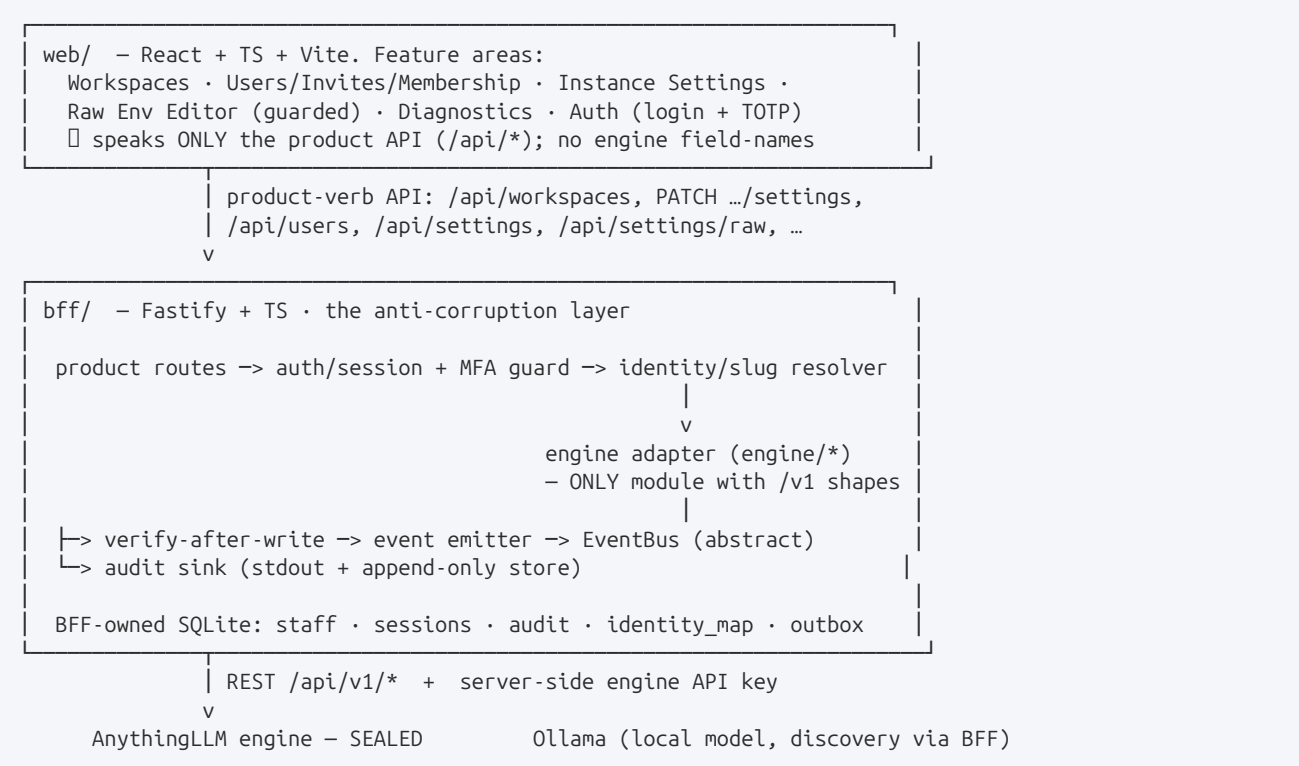
> Living document. *This is the source of truth for this app's logical and physical architecture. Update it in the same change that alters the architecture (new BFF layer or module, new persisted entity, new domain event, deployment/topology change, or a shift in how the boundary rules are enforced).* `architecture-explained.pdf` *in this directory is a generated artifact — regenerate it from this file after edits.*
>
> *Last updated: 2026-07-03 · Traces to* `specs/admin-console.md` *(rev 4),* `docs/governing-architecture.md`*, and* `docs/design/00-06`*.*

1. What this app is

A staff/operator console used by our own staff to administer a single customer's self-hosted AnythingLLM installation: its workspaces, its users, and all instance-wide settings. One deployment targets one customer's engine (one box, one customer, many users). It is the administrative counterpart to the customer-facing chat app (`front-end-custom`) and obeys the same governing white-label strategy: the engine is a sealed dependency reached only through `/api/v1`.

2. Logical architecture

Two packages plus the sealed engine and the customer's local model. The BFF is the anti-corruption layer — the only code that knows the engine's shapes.



BFF layers (dependency flows downward only)

1. Product routes — the clean, product-verb contract the web app consumes.
2. Auth/session + MFA — local staff accounts, argon2id passwords, mandatory TOTP.
3. Identity/slug resolver — maps product IDs <-> opaque engine slugs/ids (engine identifiers are never parsed; mapping lives in our SQLite `identity_map`).
4. Engine adapter (`bff/src/engine/*`) — the *sole* place engine `/api/v1` paths and field-names exist; attaches the API key; translates product <-> engine shapes.
5. verify-after-write — a generic runner: after any mutation, re-read engine state and confirm the intended outcome before reporting success (the engine has known write-consistency gaps).
6. Event emitter — emits exactly one `admin.*` domain event only after a verified write, via an

abstract EventBus.
7. Audit — every mutation and auth event to stdout + an append-only store, secrets redacted.

The product API (stable contract)

The web app consumes product verbs (e.g. POST /api/workspaces, PATCH /api/workspaces/:id/settings, POST /api/users, PATCH /api/settings, /api/settings/raw, /api/diagnostics/*, /api/models/ollama). The engine /api/v1 call each maps to is documented inside the BFF only (docs/design/02-product-api.md). No engine field-name appears in web/ — a release-blocking rule enforced by a static scan.

Domain events (`admin.*`, emitted on verified writes)

workspace.created/updated/deleted/documents_changed, workspace_user.assigned/unassigned, user.created/updated/suspended/reactivated/deleted, invite.created/revoked, instance.setting_changed, instance.provider_changed, raw_env.written. Payloads carry actor (staff id), target ids, what changed (secrets redacted), timestamp. Full schemas: docs/design/03-data-models.md and spec §14.

3. Boundary-rule conformance (governing architecture)

Rule	Enforced in
1. Talk only through `/api/v1`	`engine/adapter.ts` is the sole engine caller; no DB/file access
2. Synthesize events at our boundary	`events/emitter.ts`, invoked only after verify-after-write
3. Identities in our DB	`identity/*` + SQLite `identity_map`; engine ids are opaque handles
4. API key never in a browser	key in `config`, attached inside the adapter; browser holds only a session cookie

4. Data model (BFF-owned store)

Embedded SQLite (better-sqlite3) — appliance-appropriate, atomic, one file, no extra process. Tables: staff (argon2id hash, TOTP secret, enrollment state, recovery codes, suspended), sessions, audit_log (append-only), identity_map (product <-> engine), and event_outbox (written in the same transaction as the verify result). Details: docs/design/03-data-models.md.

5. Key cross-cutting mechanisms

- Auth + MFA — local login with a three-stage FSM (set-password → enroll-TOTP → verify); first staff account seeded from env at bootstrap, forced to set password + enroll MFA.
- Event bus (interim, decided for v1) — the shared on-box bus does not exist yet; we publish behind an abstract EventBus with a transactional outbox + in-process emitter. A relay drains the outbox to the real bus when one is configured. Adopting the real bus is a new adapter with no route/service changes. (See docs/design/06-risks.md R1.)
- Ollama model discovery — a BFF route calls Ollama's /api/tags server-side to populate model dropdowns when the effective provider is Ollama; free-text fallback otherwise and when Ollama is unreachable.
- Guarded raw-env editor — writes arbitrary accepted engine env keys behind an advanced-mode gate, whitelist (exactly the engine's accepted keys), masked diff, typed confirmation, and an audited admin.raw_env.written event.
- Secrets — GET /v1/system returns secrets as booleans; UI shows set/not-set and overwrites without revealing stored values.

6. Physical architecture

Production (target)

Runs on the same customer appliance (Mac Studio) as the engine and the customer app:

```
Staff (over LAN/VPN) ──> [ admin web (static build) ]
                         [ admin bff (Fastify) ] ──> [ AnythingLLM engine (/api/v1) ]
                              │   └─> SQLite (staff/sessions/audit/map/outbox)
                              └─> Ollama /api/tags (discovery)      └─> on-box Ollama
                         event_outbox ──(relay, when bus exists)──> shared on-box event bus
```

- Prerequisite: multi-user mode must be enabled out-of-band in the engine's native UI before user-management features work (it is session-auth-only, unreachable by the API key).
- Port allocation: the admin BFF/web share default ports (`:3002` / `:5173`) with the customer app. When co-located they must be assigned distinct ports/paths; record the real allocation here once fixed.
- Access: staff-only; expose over VPN/trusted LAN. Production CORS is restrictive (unlike the customer app's permissive dev setting).

Development

Mirrors the customer app's dev topology (WSL2 + Windows-host Ollama during dev). The engine at `:3001`, admin bff and web on their assigned dev ports.

7. Related docs

- `docs/governing-architecture.md` — the binding four-boundary-rule strategy.
- `specs/admin-console.md` (rev 4) — the requirements.
- `docs/design/00-06` — module decomposition, product API, data models, cross-cutting, web, risks.
- `docs/anythingllm-surface.md` — the real engine surface and API-key-vs-session-auth constraints.